Project Review 1

Jacob Jean 25BAI0077
Aradhya Jain 25BCE0634
Krrish Pare 25BAI0030

Title: Student University Management Systems (UMS)

Problem Statement: Existing university management architectures fail to sustain real-time transactional concurrency during peak registration cycles and lack the modularity required to adapt to evolving academic regulations, leading to service outages, seat allocation race conditions, and administrative bottlenecks.

Abstract:
Universities handle large volumes of interconnected data: student records, departments, programmes, faculty, courses, semester-wise course offerings, registrations, attendance, grades and fee payments. In many institutions this data still sits in spreadsheets or separate departmental systems. The result is duplicated records, inconsistent information, slow retrieval and errors such as students registering for courses that are full or for which they lack prerequisites. Existing solutions also lack the ability of recording hostel facilities.
This project designs and implements a centralised relational database, the **University Student Management System**, that brings this information together in one consistent schema. It models departments, programmes, students, faculty, courses, semesters, course offerings, enrollments, attendance and fee payments, with proper primary keys, foreign keys and integrity constraints. Each piece of data is stored once and linked through relationships. The system supports student admission, course registration with capacity and prerequisite checks, faculty allocation, attendance tracking, grade recording, CGPA computation and fee tracking. It also provides reports such as class lists, attendance shortage lists, department-wise performance and pending fees.

Objectives:

1. Design a **conceptual ER model** that captures all entities, attributes and relationships of university academic administration, with correct cardinalities and participation constraints.
2. Convert the ER model into a **normalised relational schema (up to 3NF)** with appropriate primary and foreign keys.
3. Enforce **data integrity** through PRIMARY KEY, FOREIGN KEY, UNIQUE, NOT NULL and CHECK constraints

4. Support core operations like **student admission, course registration, faculty allocation, attendance marking, grade entry and fee payment**
5. Modular architecture which ensures isolation between backend and frontend
6. Modern and responsive UI, even on older computer systems

Existing Systems:

- Ellucian Banner (Oracle Database backend; global enterprise standard for large universities and research systems)
- Oracle PeopleSoft Campus Solutions (Oracle Database backend; enterprise ERP for large public and state universities)
- Ellucian Colleague (MS SQL Server / MultiValue backend; standard for community colleges and mid-sized institutions)
- Workday Student (Cloud-native relational multi-tenant architecture; modern enterprise alternative to legacy on-prem systems)
- Jenzabar One / SONIS (MS SQL Server backend; widely used in mid-tier and private specialized colleges)
- SAP for Higher Education & Student Lifecycle Management (SLCM) (SAP HANA backend; global research universities)
- TCS iON Digital Campus (Oracle / Enterprise SQL backend; heavily adopted across large Indian and international university campuses)
- Camu Digital Campus (Cloud SQL / PostgreSQL backend; unified SIS and LMS platform for outcome-based education)
- Academia ERP by Serosoft (PostgreSQL / MySQL / Oracle backend; widely deployed across autonomous colleges and multi-campus institutions)
- ERPNext Education Module (MariaDB / PostgreSQL backend; open-source modular ERP for custom academic deployments)

Proposed system:

The proposed system will have a modular architecture that decouples the backend (Database, Middleware) from the frontend designed to replace brittle monolithic workflows with high performance, scalability, and seamless user experiences. Using views and privilege management, our system delivers role-tailored dashboards keeping security in mind.

USP:

Course waitlist with automatic seat allocation

- If an offering is full, the student joins a **waitlist** with a timestamp.

- When an enrolled student drops, the first student in the queue is **automatically enrolled**.
- Seats are allocated fairly on a first-come, first-served basis, and nobody has to keep refreshing.

Functional Requirements:

1. **Academic Organization & Curriculum Management:**
    - Maintain hierarchical organization across departments and degree programmes.
    - Store unified course catalog entries with designated credit values, assigned faculty instructors, scheduled terms, classroom allocations, and capacity limits.
    - Enforce course-level prerequisite rules through an attribute-level prerequisite reference prior to enrollment approval.
2. **Student Lifecycle & Admissions:**
    - Ingest and maintain student demographic, identification, program registration, and cumulative performance (CGPA) metrics.
    - Maintain historical enrollment statuses, letter grades, and earned grade point metrics per course attempt.
3. **Course Registration & Concurrency Control (USP):**
    - Provide atomic seat validation: allow enrollment only when available_seats > 0.
    - Prevent over-enrollment and double-booking during peak registration cycles.
    - Implement the **Course Waitlist Engine (USP)**:
        - Automatically place excess student requests into a timestamped, ordered FIFO queue (COURSE_WAITLIST).
        - Trigger automated seat reallocation from the top of the queue upon any student drop/cancellation event.
4. **Attendance Administration:**
    - Record and update daily, session-level student presence and absence records tagged directly across student and course vectors.
    - Calculate aggregate attendance percentages to support automated flagging of attendance shortages.
5. **Facility & Hostel Administration:**

- Map residential blocks and room inventories.
- Enforce room occupancy constraints (e.g., maximum bed capacities) upon student room allotment.

6. **Financial Operations & Billing:**
    - Track ledger charges across tuition, examinations, and hostel maintenance per semester.
    - Maintain audit logs of payments, settled balances, and pending dues.

Identification of Entities and Attributes:

Strong Entities

1. **DEPARTMENT**
    - `dept_id` *(Primary Key)*
    - `dept_name`
    - `office_location`
    - `contact_email`
2. **PROGRAMME**
    - `programme_id` *(Primary Key)*
    - `prog_name`
    - `degree_level` (e.g., B.Tech, M.Tech, Ph.D.)
    - `total_credits`
3. **FACULTY**
    - `faculty_id` *(Primary Key)*
    - `first_name`
    - `last_name`
    - `email`
    - `designation`
4. **SEMESTER**
    - `semester_id` *(Primary Key)*
    - `academic_year`
    - `term_type` (Fall, Winter, Summer)
    - `start_date`
    - `end_date`

5. **COURSE**
    - `course_id` *(Primary Key)*
    - `course_code`
    - `course_title`
    - `credits`
    - `prerequisite_code` *(Self-referencing attribute)*
    - `total_capacity`
    - `available_seats`
    - `classroom`
6. **STUDENT**
    - `student_id` *(Primary Key)*
    - `reg_no` *(Unique Key)*
    - `first_name`
    - `last_name`
    - `email`
    - `admission_date`
    - `current_cgpa`
7. **COURSE_WAITLIST**
    - `waitlist_id` *(Primary Key)*
    - `timestamp`
    - `position`
    - `status` (Waiting, Cleared, Expired)
8. **HOSTEL_BLOCK**
    - `hostel_id` *(Primary Key)*
    - `hostel_name`
    - `gender_type`
    - `warden_name`
    - `warden_contact`
9. **HOSTEL_ROOM**
    - `room_id` *(Primary Key)*
    - `room_number`
    - `capacity`
    - `occupancy`
    - `room_type` (AC, Non-AC)
10. **FEE_PAYMENT**

- `payment_id` *(Primary Key)*
- `fee_type` (Tuition, Hostel, Exam)
- `amount_due`
- `amount_paid`
- `payment_date`
- `payment_status` (Pending, Partial, Completed)

Weak Entity

- **`ENROLLMENT`**
  - **Identifying Owner Entities:** `STUDENT`, `COURSE`
  - **Key Structure:** Composite Primary Key derived from owners `(student_id, course_id)`
  - **Partial Key / Discriminator Attributes:** `enrollment_date`, `status`
  - **Descriptive Attributes:** `grade_letter`, `grade_points`
- **`ATTENDANCE`** (Modeled as an associative relationship between `STUDENT` and `COURSE`):
  - `attendance_date`
  - `status` (Present, Absent, Excused)
  - `marked_by_fac`

Primary Keys and Relationships:

Relationship Name	Participating Entities	Relationship Type	Foreign Key Placement (Logical Relational Mapping)
Offers	`DEPARTMENT` : `PROGRAMME`	Standard 1:N	`PROGRAMME.dept_id` → `DEPARTMENT.dept_id`
Employs	`DEPARTMENT` : `FACULTY`	Standard 1:N	`FACULTY.dept_id` → `DEPARTMENT.dept_id`
Owns	`DEPARTMENT` : `COURSE`	Standard 1:N	`COURSE.dept_id` → `DEPARTMENT.dept_id`
Enrolled_In	`PROGRAMME` : `STUDENT`	Standard 1:N	`STUDENT.programme_id` → `PROGRAMME.programme_id`

Relationship Name	Participating Entities	Relationship Type	Foreign Key Placement (Logical Relational Mapping)
Teaches	FACULTY : COURSE	Standard 1:N	COURSE.faculty_id → FACULTY.faculty_id
Scheduled_In	SEMESTER : COURSE	Standard 1:N	COURSE.semester_id → SEMESTER.semester_id
Takes	STUDENT : ENROLLMENT	Identifying 1:N	ENROLLMENT.student_id → STUDENT.student_id
Includes	COURSE : ENROLLMENT	Identifying 1:N	ENROLLMENT.course_id → COURSE.course_id
Attendance	STUDENT : COURSE	Associative M:N	ATTENDANCE(student_id, course_id, attendance_date)
Has_Waitlist	COURSE : COURSE_WAITLIST	Standard 1:N	COURSE_WAITLIST.course_id → COURSE.course_id
Joins	STUDENT : COURSE_WAITLIST	Standard 1:N	COURSE_WAITLIST.student_id → STUDENT.student_id
Contains	HOSTEL_BLOCK : HOSTEL_ROOM	Standard 1:N	HOSTEL_ROOM.hostel_id → HOSTEL_BLOCK.hostel_id
Allocated_To	HOSTEL_ROOM : STUDENT	Standard 1:1 (Partial)	STUDENT.room_id → HOSTEL_ROOM.room_id

Relationship Name	Participating Entities	Relationship Type	Foreign Key Placement (Logical Relational Mapping)
Pays	`STUDENT` : `FEE_PAYMENT`	Standard 1:N	`FEE_PAYMENT.student_id` → `STUDENT.student_id`
Billed_For	`SEMESTER` : `FEE_PAYMENT`	Standard 1:N	`FEE_PAYMENT.semester_id` → `SEMESTER.semester_id`

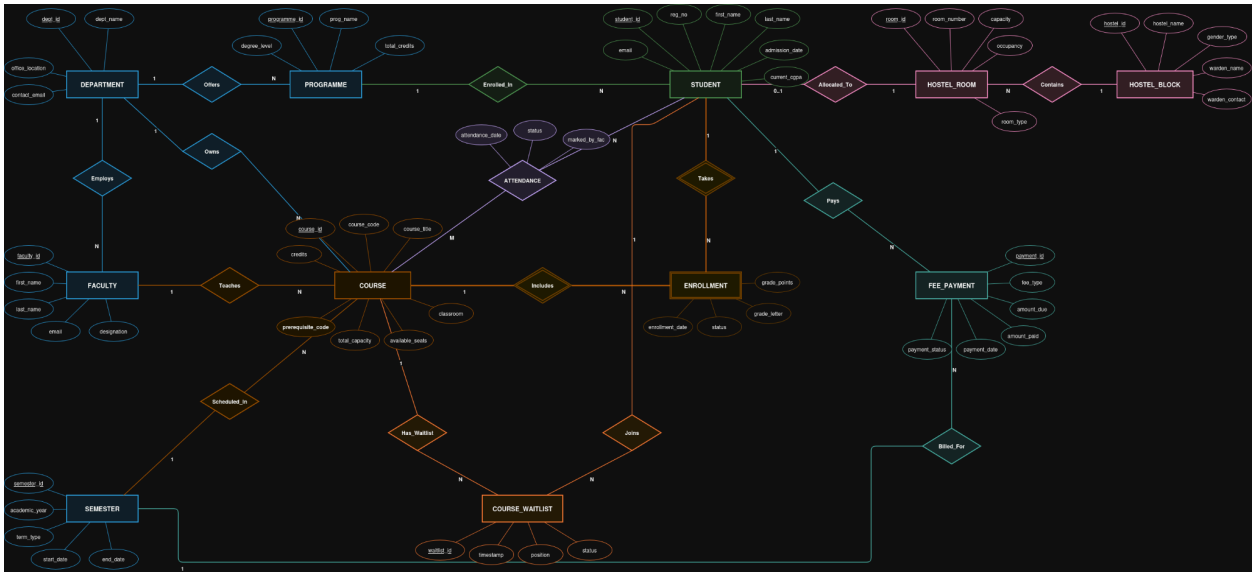
Cardinality:

Relationship	Ratio	Min/Max Constraints	Meaning
`DEPARTMENT` → `PROGRAMME`	$1:N$	$(1,N):(1,1)$	A department must offer at least one programme; a programme belongs to exactly one department.
`DEPARTMENT` → `FACULTY`	$1:N$	$(1,N):(1,1)$	A department employs one or more faculty; a faculty member belongs to exactly one department.
`DEPARTMENT` → `COURSE`	$1:N$	$(1,N):(1,1)$	A department owns multiple courses; a course is owned by one department.
`PROGRAMME` → `STUDENT`	$1:N$	$(0,N):(1,1)$	A programme may have zero or many students; a student must be enrolled in exactly one programme.

Relationship	Ratio	Min/Max Constraints	Meaning
`FACULTY` → `COURSE`	$1:N$	$(0,N):(1,1)$	A faculty member may teach zero or multiple courses; a course section has one primary instructor.
`SEMESTER` → `COURSE`	$1:N$	$(0,N):(1,1)$	A semester runs multiple courses; a course entry is scheduled in a specific semester.
`STUDENT` → `ENROLLMENT`	$1:N$	$(0,N):(1,1)$	Total participation on weak entity `ENROLLMENT`; a student can have multiple enrollments.
`COURSE` → `ENROLLMENT`	$1:N$	$(0,N):(1,1)$	Total participation on weak entity `ENROLLMENT`; a course section contains multiple enrollments.
`STUDENT` → `COURSE` *(Attendance)*	$M:N$	$(0,N):(0,N)$	Direct associative tracking of attendance across students and course lectures.
`COURSE` → `COURSE_WAITLIST`	$1:N$	$(0,N):(1,1)$	A course may maintain multiple waitlisted entries; each waitlist record belongs to one course.
`STUDENT` → `COURSE_WAITLIST`	$1:N$	$(0,N):(1,1)$	A student may waitlist for zero or multiple courses; each entry is tied to one student.

Relationship	Ratio	Min/Max Constraints	Meaning
HOSTEL_BLOCK → HOSTEL_ROOM	$1 : N$	$(1, N) : (1, 1)$	A hostel block contains multiple rooms; a room belongs to one hostel block.
HOSTEL_ROOM → STUDENT	$1 : 1$	$(0, 1) : (0, 1)$	Partial on both sides: a student may be a day scholar; a room bed slot can be unoccupied.
STUDENT → FEE_PAYMENT	$1 : N$	$(0, N) : (1, 1)$	A student makes multiple fee payments; each payment record belongs to one student.
SEMESTER → FEE_PAYMENT	$1 : N$	$(0, N) : (1, 1)$	A semester bills multiple payment records; each fee is billed for a specific semester.

ER Diagram:



Explanation of the ER Diagram:

1. **Academic Organization Backbone:**
    - The hierarchy starts with `DEPARTMENT`, which acts as the institutional parent entity. It maintains 1-to-many relationships with `PROGRAMME` (curricula offered), `FACULTY` (professors employed), and `COURSE` (catalog modules owned). This structure eliminates record duplication across departments.
2. **Consolidated Course Entity:**
    - Instead of splitting courses into abstract catalog entries and fragmented offering instances, `COURSE` models the semester offering directly. It includes scheduling and operational attributes (`classroom`, `total_capacity`, and `available_seats`).
    - Prerequisites are maintained within the entity through `prerequisite_code`, an attribute linking back to a parent `course_code`. This avoids an extra junction table and simplifies prerequisite lookups.
3. **Weak Entity Modeling for `ENROLLMENT`:**
    - `ENROLLMENT` is modeled as an **identifying weak entity** that depends on both `STUDENT` and `COURSE`.
    - An enrollment record has no independent existence without a student registered in a course section.

- Its primary key is composite: `(student_id, course_id)`. In the ER diagram, this is drawn with a **double-bordered rectangle** linked to double-bordered relationship diamonds (`Takes` and `Includes`), enforcing total existence dependency.

4. **Direct Associative Attendance Tracking:**
    - Daily class attendance is modeled as an associative relationship (`ATTENDANCE`) between `STUDENT` and `COURSE`.
    - It captures date-stamped records (`attendance_date`, `status`, `marked_by_fac`) directly across student and course entities. This allows faculty to take attendance and verify attendance thresholds without running queries across the enrollment ledger.
5. **Waitlist Automation (Project USP):**
    - The `COURSE_WAITLIST` entity provides fair, first-come, first-served queue processing.
    - When a transaction finds that `COURSE.available_seats == 0`, it adds a record to `COURSE_WAITLIST` with a precise `timestamp` and ordinal `position`.
    - When an enrolled student drops a course, a database trigger or event bus queries the earliest active waitlist record (`ORDER BY timestamp ASC LIMIT 1`), converts it into a new `ENROLLMENT` record, and decrements `available_seats` atomically. This prevents race conditions and eliminates the need for manual retries during peak registration periods.
6. **Hostel Facilities Integration:**
    - The design includes `HOSTEL_BLOCK` and `HOSTEL_ROOM` to address a gap in legacy student systems noted in the project review.
    - Rooms are linked to blocks via a 1:N `Contains` relationship. The relationship between `HOSTEL_ROOM` and `STUDENT` (`Allocated_To`) is optional 1:1, allowing the system to represent day scholars (who have no room assignment) while tracking room occupancy.
7. **Financial Integrity & Fee Auditing:**
    - `FEE_PAYMENT` connects `STUDENT` and `SEMESTER`.
    - Every invoice tracks the specific `fee_type` (Tuition, Hostel, Exam), `amount_due`, `amount_paid`, and `payment_status`. This allows the registrar to check whether fees are clear before validating semester course registrations.